Algoritmos e Programação de Computadores

Repetição

Prof. Daniel Sundfeld Lima
daniel.sundfeld@unb.br

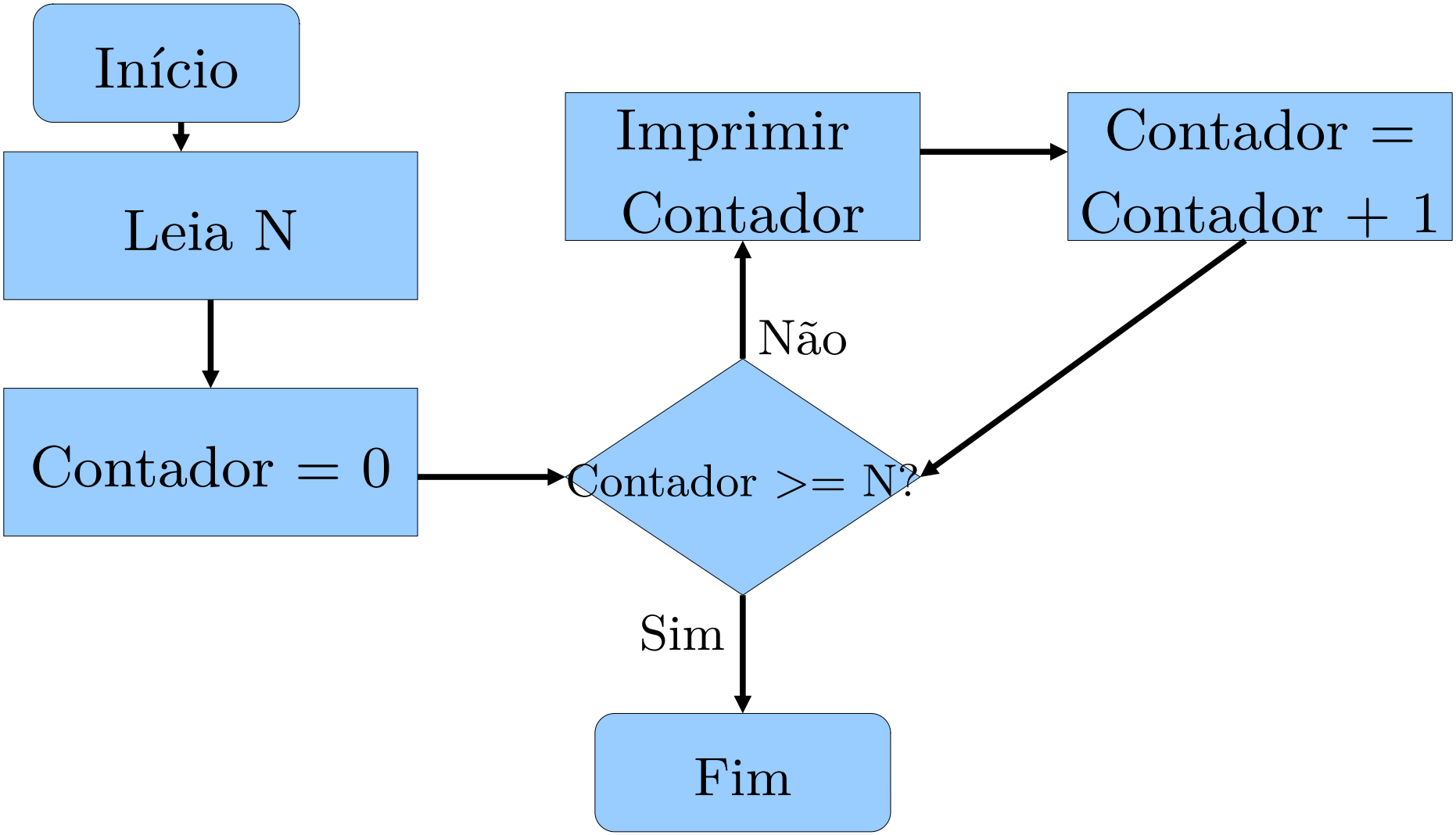
Introdução

- Vimos que as condicionais são utilizadas para determinar múltiplos fluxos de execução no seu código
- Desta um programa pode determinar se um conjunto de instruções deve ou não ser executado
- Essa estrutura de repetição é básica na programação e chamada de **laço**

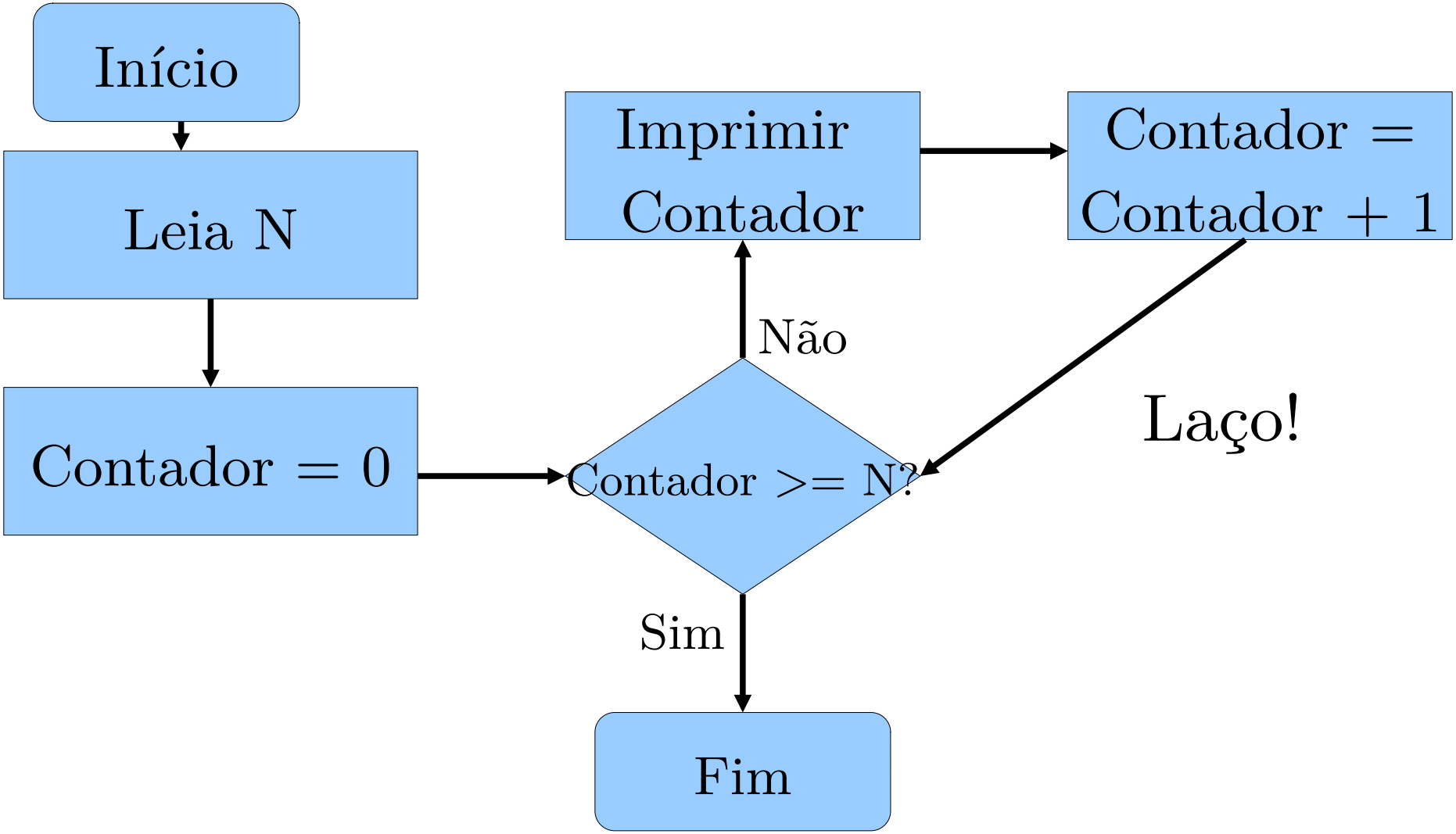
Introdução

- Durante a construção de programas é comum repetirmos as instruções que serão precisam ser executadas
- Por exemplo: dado um número N, imprimir todos os números de 0 até N

Fluxograma


Início
Leia N
Contador = 0
Contador >= N?
Não
Imprimir Contador
Contador = Contador + 1
Sim
Fim

Fluxograma


Início
Leia N
Contador = 0
Contador >= N?
Não
Imprimir Contador
Contador = Contador + 1
Laço!
Sim
Fim

While

- Para construir esse código em C, podemos utilizar a palavra-chave: **while** (enquanto)
- Sintaxe:
  - `while (condição) { ... }`
- As instruções `{ ... }` serão executadas enquanto a `condição` for verdadeira

```c
#include <stdio.h>

int main()
{
    int N;
    int contador = 0;

    scanf("%d", &N);

    while (contador <= N)
    {
        printf("%d\n", contador);
        contador = contador + 1;
    }
    return 0;
}
```

```c
#include <stdio.h>

int main()
{
    int N;
    int contador = 0;

    scanf("%d", &N);

    while (contador <= N)
    {
        printf("%d\n", contador);
        contador = contador + 1;
    }
    return 0;
}
```

Não se usa ;

Do / While

- Também é possível usar a estrutura **do/while**
- Sintaxe:
  - `do { ... } while (condição)`
- As instruções `{ ... }` serão executadas enquanto a `condição` for verdadeira
- Qual seria a diferença para o **while**?

Comparação while <-> do / while

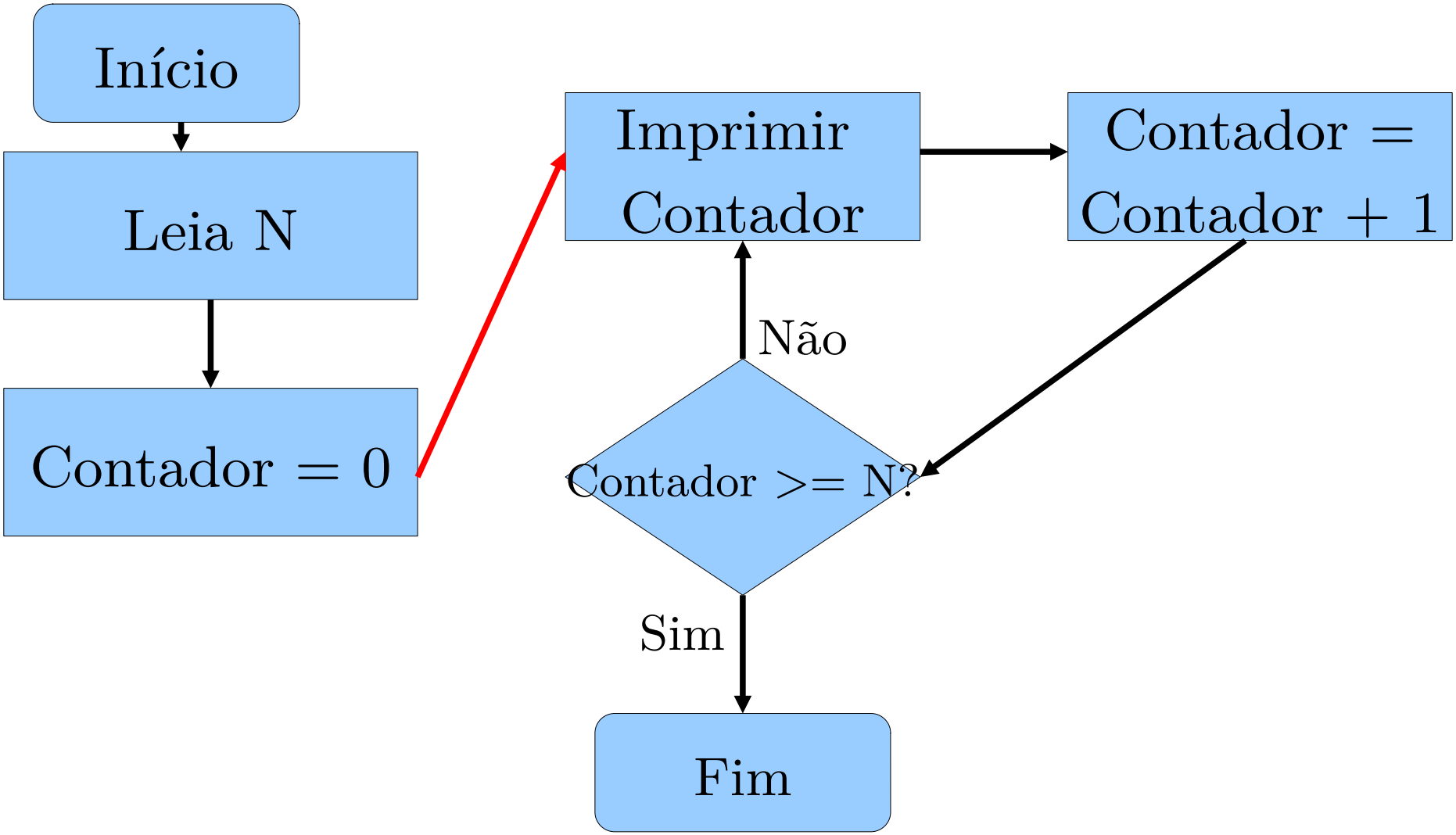
```
while (condição) { ... }
```

- Verifica a condição -> Executa instruções

```
do { ... } while (condição)
```

- Executa instruções -> Verifica condição
- O do/while executa pelo menos uma vez as instruções, mesmo que a (condição) seja sempre falsa!

Comparação while <-> do / while



Comparação while <-> do / while

Fonte: https://www.reddit.com/r/ProgrammerHumor/

Comparação while <-> do / while

```
int main()
{
    int N;
    int contador = 0;

    scanf("%d", &N);

    do
    {
        printf("%d\n", contador);
        contador = contador + 1;
    }
    while (contador <= N);

    return 0;
}
```

Comparação while <-> do / while

```
int main()
{
    int N;
    int contador = 0;

    scanf("%d", &N);

    do
    {
        printf("%d\n", contador);
        contador = contador + 1;
    }
    while (contador <= N);

    return 0;
}
```

Usa-se ;

Comparação while <-> do / while

- Os dois códigos que mostramos anteriormente, possuem o mesmo comportamento com **while** e **do / while** para muitas entradas
- Nem sempre isso não é verdade

Comparação while <-> do / while

```c
int main()
{
    int N;
    int contador = 0;

    scanf("%d", &N);

    while (contador < N)
    {
        printf("%d\n", contador);
        contador = contador + 1;
    }
    return 0;
}
```

```c
int main()
{
    int N;
    int contador = 0;

    scanf("%d", &N);

    do
    {
        printf("%d\n", contador);
        contador = contador + 1;
    }
    while (contador < N);

    return 0;
}
```

Comparação while <-> do / while

- Neste exemplo, mesmo com a modificação, a execução é semelhante
- Se o usuário digitar o número 0 (ou negativo) serão diferentes
- Digitando 0:
  - Código while: não imprime nada
  - Código do/while: imprime 0

For

- Outra palavra chave muito importante é o **for** (para cada)
- Ele é especialmente utilizado para contadores, mas pode ser usado genericamente para vários laços complexos

For

- Veja o seguinte código:

```c
#include <stdio.h>

int main()
{
    int i;

    i = 0;
    while (i < 10)
    {
        printf("%d\n", i);

        i++;
    }

    return 0;
}
```

Nota:
Programadores começam a contar no 0 normalmente...

For

- Esse código pode ser dividido nas partes:
  - Inicialização
  - Condição
  - Laço
  - Incremento

```
#include <stdio.h>

int main()
{
    int i;

    i = 0;
    while (i < 10)
    {
        printf("%d\n", i);

        i++;
    }

    return 0;
}
```

For

- Essa estrutura de repetição se repete diversas vezes, que pode ser simplificado utilizando um laço **for**

```
int main()
{
    for (int i = 0; i < 10; i++)
    {
        printf("%d\n", i);
    }
    return 0;
}
```

For

Inicialização

Condição

Incremento

```
int main()
{
    for (int i = 0; i < 10; i++)
    {
        printf("%d\n", i);
    }
    return 0;
}
```

Laço

For

- No **for**, é opcional fazer a declaração da variável no laço, ou utilizar uma que já existe

```c
int main()
{
    for (int i = 0; i < 10; i++)
    {
        printf("%d\n", i);
    }
    return 0;
}
```

```c
int main()
{
    int i;
    for (i = 0; i < 10; i++)
    {
        printf("%d\n", i);
    }
    return 0;
}
```

"Old-fashion"

For

- Podemos utilizar alguns membros vazios
- Para ser vazio, continuamos usando “;” dentro do for, mas não colocamos nada

```
int main()
{
    int i;

    scanf("%d", &i);
    for ( ; i < 10; i++)
    {
        printf("%d\n", i);
    }
    return 0;
}
```

For

- Também é possível utilizar múltiplos campos, separando-os por vírgulas:

```
int main()
{
    for (int i = 0, j = 10 ; i < 10; i++, j --)
    {
        printf("%d - %d\n", i, j);
    }
    return 0;
}
```

```
Saída do código:
0 - 10
1 - 9
...
9 - 1
```

Laço Infinito

- O loop infinito é um problema causado quando a condição de um loop nunca é falsa

```
int main()
{
    int i = 0;
    int j = 5;

    while (i < j)
    {
        printf("i e' menor do que j\n");
    }
    return 0;
}
```

Laço Infinito

- Neste exemplo, o programa irá imprimir indefinidamente a mensagem na tela e nunca irá parar por conta própria (return 0 na main)

Aninhamento

- Os loops também pode ser aninhados
- Os exemplos que vimos em sala são simples, mas códigos complexos podem demandar vários loops dentro do outro!

Aninhamento

```c
int main()
{
    for (int i = 1; i < 10; i++)
    {
        for (int j = 0; j < i; j++)
        {
            printf("%d ", j);
        }
        printf("\n");
    }
    return 0;
}
```

Aninhamento

- Saída do código anterior:

```
0
0 1
0 1 2
0 1 2 3
0 1 2 3 4
0 1 2 3 4 5
0 1 2 3 4 5 6
0 1 2 3 4 5 6 7
0 1 2 3 4 5 6 7 8
```

Exercício

- Faça um código que leia N duplas de número e imprime a soma deles. A primeira linha contém o número de casos

3

1 2

2 3

3 3

Exercício

```c
#include <stdio.h>

int main()
{
    int N;
    int x, y;

    scanf("%d", &N);

    for (int i = 0; i < N; i++)
    {
        scanf("%d %d", &x, &y);
        printf("%d\n", x + y);
    }
    return 0;
}
```

Dúvidas?

- daniel.sundfeld@unb.br